



SymPy

Google Summer of Code 2026

Enhancing Assumptions via EUF Integration, LRA Robustness,
and Property-Based Testing

Sujal Jayant Kumar

Indian Institute of Technology Madras
B.Tech in AI and Data Analytics (2024-2028)
GitHub ID: [SujalKumar06](#)

March 31, 2026

Potential Mentors:

Tilo RC, Aaron Meurer

1 Me the person

Name: Sujal Jayant Kumar

University: Indian Institute of Technology Madras (IITM)

Degree: B.Tech (Bachelor of Technology)

Major: AI and Data Analytics

Residence: Chennai, India

Timezone: Indian Standard Time (UTC+5:30)

GitHub: [SujalKumar06](#)

Email: sujaljayantkumar06@gmail.com, da24b026@smail.iitm.ac.in

Hey, I am Sujal, a second year undergraduate currently pursuing B.Tech in AI and Data Analytics in IIT Madras. It is a great pleasure to study in IIT Madras, which currently has [National rank 1](#) in India.

My hobbies are making jokes, performing stand up comedy, and watching sports like cricket and F1. Most of my time during the day is spent on GitHub understanding and fixing issues. I primarily work on Ubuntu 24.04.4

I personally want to participate in GSoC with SymPy to improve my programming skills, and I want to contribute to a project that serves the global scientific community. There is immense satisfaction in building open-source tools whose impact extends far beyond what any of us could achieve individually.

Courses that I have completed or am currently taking in my college are:

- Algorithms for Data Science (Studied SAT problems and solvers)
- Computational Physics and Computational Methods for Data Science (Both being heavy use cases of math and numerical linear algebra)
- Programming and Data Structures, Computer Systems (Programming in Python)
- Math courses like Calculus, Linear Algebra, Probability, and Statistics.

2 Me the programmer

Editor

- I use VS Code for most of my jobs. It is my primary editor. I have experience with Jupyter Notebook for Python notebooks, but I prefer VS Code for that job as well.
- I really like VS Code because of the number of extensions it provides; for any work, you can find an extension available. I am quite familiar with VSC's Python debugger, which helps me a lot in the debugging process.

Programming Experience

- I have been programming for 2 years now. I am highly comfortable with Python and also proficient in C++. I mostly do my tasks (programming ones) in Python.

- I also contribute to `matplotlib`; this is done as a group of people working together on the same issue. One of the recent and major PR we raised was [PR #30811](#).
- I am currently part of The Programming Club at IITM. (www.progclubiitm.com) (I was involved in the making of this website). As part of the club, we teach students about software development and competitive programming (CP). We even conduct national level competitions and make questions for them.
- A friend and I are currently building our own Python interpreter from scratch using C++ under the Programming Club.
- My coursework in AI and Data Analytics is highly project-driven, requiring the rigorous application of machine learning concepts. This has helped me become proficient with Python and the external packages used in the projects.
- Apart from spending my whole day on GitHub, I also participate in Competitive Programming Contests on [Codeforces](#) and solve questions.

More on Python

As mentioned earlier, I am quite proficient in the Python language. Few of my favorite features in python are:

- Arbitrary-precision arithmetic means that there is basically no overflow limit for integers in Python; it keeps adding more memory to support more precision. In C++, `int` just overflows after a point and you should always keep your eyes open for it.

- ```
temp = 0
for i in range(3):
 temp = i
else:
 print("Hi")
```

You can write else blocks for for loops! The `else` block is only executed if the for loop ends naturally and not through any breaks. If you have to do the same thing in C++, you might need to declare flags for breaks.

- ```
print(1 and 2) # Returns 2
```

Python returns `b` while evaluating `a and b` if `a` is truthy and `b` if `a` is falsy.

The most advanced standard library functionality I have used is the `dis` module. Using this module, you can see and analyze the opcodes of functions, observe the stack operations, and understand what python does under the hood. One cool thing I learned while using this was that list comprehensions are faster than making a list and then populating it with numbers using the `append` function.

SymPy Fav Feature!!

I really like the `srepr` function. It helps you see the AST underneath. This also gave me a small idea of how ASTs actually work and how to traverse them properly. This is quite interesting for a programmer to understand how the Math he/she writes on the program, is being executed.

Git Experience

I have been using Git from the start of my college (approximately 2 years), so I have a good idea of Git functionalities. I have been opening and working on different PRs, which makes my hold on Git much stronger.

For common tasks like staging changes and committing, I mostly use VS Code's source control. For tasks like rebasing, bisecting, or squashing, I use terminal/git commands.

I have even conducted a session in my college with my colleague, teaching the general student body about git.

3 Contributions to SymPy

All of these are with respect to the date March 31, 2026.

PRs

- [Merged] [PR #28235](#) - Added `__format__` method to rational
- [Closed] [PR #28379](#) - Updating Documentation of Symbol. (Was continued by [PR #28889](#) and that PR got merged.)
- [Merged] [PR #28873](#) - Add: Added `refine_sin_cos` function to handle simplifications for cos and sin
- [Merged] [PR #28967](#) - parsing: Stripped extra 1, while parsing unevaluated fractions
- [Merged] [PR #28967](#) - series: Pass `dir` to recursive limit calls in `pow_heuristics`
- [Merged] [PR #28995](#) - Fix: Fixes Doc Build Error because of `sphinx>=9.0.0`
- [Merged] [PR #29022](#) - Add: Added a variable to display all old assumptions.
- [Merged] [PR #29200](#) - Fix: `ask(Q.real(sqrt(x)), Q.real(x))` should return None
- [Merged] [PR #29225](#) - assumption: Fix `NonZeroPredicate` for the `Pow` function.
- [Merged] [PR #29457](#) - assumptions: Fix inconsistent assumptions in `satask` for `0*irrational`
- [Open] [PR #29533](#) - assumptions: Update `refine_sin_cos` to handle more simplifications

- [Open] [PR #29481](#) - assumptions: Fix handlers for Mul of 0 with irrational/imaginary numbers
- [Draft] [PR #29120](#) - assumptions: Add helper function to improve order predicates for Add.
- [Closed] [PR #28860](#) - Fix: ask(False, False) returns False instead of raising ValueError
- [Closed] [PR #28379](#) - Shifting rad and deg functions

Issues

- [Closed] [Issue #29096](#) - ask($x > -1$, Q.positive(x)) returns None instead of True.
- [Closed] [Issue #29433](#) - ask incorrectly raises Inconsistent assumptions instead of None.
- [Open] [Issue #29480](#) - ask raises incorrect result for Mul 0 with irrational/imaginary numbers
- [Open] [Issue #29114](#) - ask returns None for Q.nonzero(exp(x)) given Q.real(x)
- [Open] [Issue #29143](#) - ask fails to deduce properties of n from assumptions on -n

There were few PRs in which I was involved in the review/contribution process.

Reviews/Contribution

- [Open] [PR #29146](#) Fix NonZeroPredicate for proposition on exp(x)
- [Closed] [PR #29150](#) Permutation subgroup leaving a list of polynomials invariant
- [Open] [PR #29131](#) Add refine.log handler for logarithm simplifications
- [Closed] [PR #29110](#) Fix odd predicate behavior under negation
- [Closed] [PR #29108](#) fixed LRASolver failing on implied real assumptions
- [Closed] [PR #29099](#) fixed LRASolver failing on implied real assumptions

4 Project Proposal

Overview of the Project

From here I will talk about my project proposal. I would like to divide my project into 3 phases to give everyone a proper sense of clarity regarding my thoughts. I intend to work on a 350-hour project.

- My proposal in for the `assumptions` module. In the years 2023 and 2025, two impactful GSoC projects were conducted on the assumptions module. These projects introduced advanced mathematical satisfiability models to SymPy, specifically Linear Real Arithmetic (LRA) SAT solvers, a Congruence Closure Engine, and a DPLL(T) framework.
- The major issue with the last 2 was not with how mathematically strong they are, but with the current open PRs that are trying to integrate this. However, these are very massive, and the single implementation has made them incredibly difficult for collaborators to review. These algorithms cannot currently be used by end-users.
- My proposal essentially aims to be the bridge that brings those experimental solvers into SymPy's core functionality. Instead of building in isolated algorithms, the primary objective is Integration and Stabilization of these algorithms.
- Although the LRA SAT Solver was successfully merged, it left behind some structural issues. Right now, even simple queries trigger a massive number of slow, redundant SAT evaluations. A major cause of this is the recursive handlers, which often lead to infinite loops and repeated `satask` calls. My main goal is to decouple these recursive handlers from the core API and add proper preprocessing steps for the LRA theory.
- All of the unit tests in the assumptions module are static, which cannot, of course, catch every logical edge case in a SAT solver. A proper way would be to introduce property-based testing and fuzzing via the `Hypothesis` library in Python to automatically find regressions and thus fix the bugs along the way and make the solvers more robust.

What has been already done!?

Tilo RC's Project in 2023 for Linear Real Arithmetic (LRA).

- SymPy was enhanced to process linear inequalities by implementing a Linear Real Arithmetic (LRA) SAT solver. LRA is about checking whether a bunch of linear inequalities/equalities (without any quantifiers like $\forall$ or $\exists$), combined using AND/OR/NOT (Boolean Operators), can all be true at the same time. These constraints take the general form:

$$\sum_{i=1}^n c_i x_i \bowtie d$$

where $c_i, d \in \mathbb{R}$, x_i are real-valued variables, and $\bowtie \in \{<, \leq, =, \geq, >\}$.

- The LRA solver allows the assumptions engine to reliably process systems of inequalities, deducing geometric truths such as $(x > y) \wedge (y > z) \implies (x > z)$ (given x, y, z are defined as real as per old assumptions), and integrating this continuous mathematical logic directly into the boolean satisfiability chain.
- For example

```
from sympy import symbols, ask
x, y, z = symbols("x␣y␣z", real=True)

ask(x + y > 2, (x < 0) & (y < 0)) # returns False instead of
None
```

Krishnav Bajoria's Project in 2025 for the DPLL(T) Framework and EUF Theory.

- SymPy's architecture was further expanded to handle Equality and Uninterpreted Functions (EUF). Standard SAT solvers operate purely on booleans and cannot natively track mathematical equivalences. To resolve this, two major components were introduced in PRs [PR #28423](#) and [PR #28311](#)
- It involves a robust, modular implementation of the congruence closure algorithm for EUF, grounded in the foundational DPLL(T) architecture (Ganzinger et al., 2004) and the fast congruence closure algorithms outlined by Nieuwenhuis and Oliveras (2003).
- For example:

```
from sympy import *

x, y = Symbol('x'), Symbol('y')
f = Function('f')

print(ask(Q.prime(f(x)), Q.prime(f(y)) & Q.eq(x, y)))
#Outputs None, should output True
```

Phase 1: Reviving and Integrating the EUF Theory Solver

Essentially, Phase 1 would be reviving the PRs [PR #28423](#) and [PR #28311](#), but these are exceptionally big PRs, which makes the task of reading and reviewing very difficult for both me and the reviewers.

A way around is one suggested by Tilo in the [comment](#), the suggestion being:

One idea I just had is that we could approach reviving the PRs more collaboratively. I could be the one to actually make a new PR reviving them and do most of the work. You would focus on reviewing my work by writing tests and looking for bugs.

I liked the idea very much because this will speed up the work a lot; I should know about the algorithms and their API calls, which I will try to focus on during the Pre-GSoC period. I will talk about it later.

I aim to divide this big PR into two PRs; let's call the initial PR PR1 and the next one PR2. I will start by providing more information on what I intend to do in PR1:

- This PR will focus strictly on the math part of the framework, much of which Krishnav already completed in his PR. However, before the dense algorithm is even written, I would like to take this opportunity to properly define the architecture (the structure of the framework) once again before we start putting in the math.
- The testing of this should follow the modern **Hypothesis** testing framework, which SymPy is currently wanting to have property based testing rather than static tests. (I will talk more about this in Phase 3)
- The first step in this would be extracting all test files and core structure from #28423. According to the reviews, first task is to remove the **Z3Comparator** class.
- Before I write the tests, I have to define a proper structure of `sympy/logic/algorithms/euf_theory.py`. This will not bloat the initial PR size; it would just be stubs of classes and their methods. For example:

```
class EUFCongruenceClosure:

def __init__(self, equations):
    self.pending_unions = deque()
    self.representative_table = {} ... # more things

def _register(self, const):
    pass

def _find(self, const)::
    pass

def _union(self, a, b):
    pass
```

- Once it is done, Tilo can work on the math for the algorithms and implement it in the same PR (PR1), lets jump to testing the math!
- With the stubs in place, I will replace the brittle static assertions of the PR with a robust, modern testing contract using **Hypothesis**. I will talk about this more in Phase 3 of the proposal. All the tests would be though **@XFAIL**, as Tilo keeps pushing more math in the given structure, it will pass and check if the math is technically sound.

The next PR (PR2) is where all the math has to be linked to SAT solver. This is essentially what Krishnav mentioned in his [future work](#).

This PR will only focus on wiring it correctly. The final goal should be to build a translational layer that links the SAT solver to the E-graph, wire the core DPLL(T) hooks into the backend, and use the completed architecture via a clean public API. A little brief on PR2:

- Now assuming the PR1 is complete and has been merged, linking it with SAT solver can be done by mirroring the established LRA Solver. The established LRA Solver

API contract can be found in `sympy/logic/algorithms/lra_theory.py` written by Tilo.

- **Building `sympy/logic/algorithms/euf_theory_solver.py`:** I intend to implement `EUFTTheorySolver` interface. This middleman acts as the universal translator between the boolean SAT integer guesses and the symbolic math operations executed by the E-graph.
- **Initialization and Preprocessing:** Just like in `lra_theory.py`, the SAT formula must be pre-processed before the solver starts guessing. I will implement `from_encoded_cnf` as a static method. It will scan the `EncodedCNF` mapping, extract all `Q.eq` and `Q.ne` predicates, and build the bidirectional `enc_to_expr` dictionary.
- **`assert_lit`:** When the SAT solver guesses a literal, this method translates it back into a SymPy expression. It writes down the current E-graph state to the `history_stack`, then pushes the equality to the E-graph. If the E-graph detects a geometric contradiction, it generates a conflict and returns it.
- More methods and relevant things would be implemented with proper discussion while implementing things properly.

```
class EUFTTheorySolver:
    def __init__(self, enc_to_expr):
        self.engine = EUFCongruenceClosure()
        self.enc_to_expr = enc_to_expr # Mapping: Int ->
            SymPy Equation
        self.history_stack = [] # States history

    @staticmethod
    def from_encoded_cnf(encoded_cnf):

        enc_to_expr = {}
        conflicts = []
        # Logic goes here
        return EUFTTheorySolver(enc_to_expr), conflicts

    def assert_lit(self, enc_constraint):
        # Logic similar to assert_lit in LRA Solver
        return None
```

- Because we are just doing things as LRA Solver, wiring it to SAT engine is pretty easy. This just involves us to put a flag `use_euf_theory=False` in `dp11_satisfiable`.
- I think it is good to integrate `euf.solver` directly with the `ask` instead of having an `euf_ask`. Basically natively wiring the EUF capabilities into SymPy's global `ask()` function.

```
def ask(proposition, assumptions=True, context=
    global_assumptions):
    # ... existing setup code ...
```

```
res = _ask_single_fact(key, local_facts)
if res is not None:
    return res

res = key(*args)._eval_ask(assumptions)
if res is not None:
    return bool(res)

res = satask(proposition, assumptions=assumptions,
             context=context)
if res is not None:
    return res

try:
    res = lra_satask(proposition, assumptions=assumptions
                    , context=context)
    if res is not None:
        return res
except UnhandledInput:
    pass

try:
    res = euf_satask(proposition, assumptions=assumptions
                    , context=context)
except UnhandledInput:
    return None

return res
```

This is all what I intend to do in Phase 1, I estimate putting in 4 weeks in for Phase 1.

Phase 2: Decoupling Bottlenecks & Expanding LRA Robustness

The LRA Solver was merged which has a very powerful math backend but the frontend feeding it with stuff is flawed. Even simple queries trigger a massive recursions of slow, redundant SAT evaluations. I want to address this issue first, related PR: [PR #29507](#)

- Before we start working on [#29507](#), one obstacle for us there is [PR #29506](#), [#29506](#) basically has implemented to handle most linear inequality assumptions.
- A lot of implementation has already been done, but there is a possibility that this could have bugs unnoticed. My sole work would be to assist Tilo with his implementation, I assume a lot has been already written.
- My task here is to write tests the new `Hypothesis` and static testss testing to find bugs in the PR. If found, this would help us identify potential bugs in the PR and then fix them.

- Assuming #29506 is in, we can start out work on #29507 also. The major task in hand is to bypass the intermediate recursive calls. From what I understand #29507 already has a prototype of `recursive_ask` designed to bypass the main dispatcher. However, it is currently isolated, and CNF consistency checks have been disabled.
- My main job is to finalise this structure and properly define `recursive_ask` before replace all internal ask calls by recursive_ask calls. Initially, rather than letting consistency checks trigger the full, heavy SAT solver, I will implement a lightweight, localized consistency check (which is currently commented) using the `local_facts` CNF before routing to recursive handlers.
- The same task of having Hypothesis testing would be done here, thus eliminating any edge cases or possible flaws in the logic in the code changes. This will expose us to new regressions that can be fixed.

Another issue I wanted to fix in this Phase is one from [Tilo's Final Report](#) in 2023. [Handle more type of inputs](#). I thought of working on [Implement speed improvements from paper](#) but I feel the goal of my GSoC project is to first make the existing things robust and not keep adding more things if not necessarily required.

- Currently, if the LRA Solver engine encounters an input outside its exact expected domain—such as a complex variable, an infinity (∞), or an irrational number, it immediately raises an `UnhandledInput` exception and aborts the entire satask pipeline.
- `lra_theory` strictly requires real variables. Queries like `ask(Q.gt(x, 2))` currently return `None` because the solver immediately runs away when it cannot guarantee x is real.
- We can just update the `LRAVariable` class to track the realness of a variable if, in the proposition, the SAT solver tries to assert anything related to being imaginary, a conflict should be created and returned.

```
class LRAVariable:
    def __init__(self, var):
        self.var = var
        # ... existing bounds tracking ...
        # new realness checkers
        self.is_real = None
        self.real_assertion_literal = None

class LRASolver:
    def assert_lit(self, enc_constraint):
        # ... existing logic ...

        # New logic, if a conflict is caused
        # because of something being asserted as imaginary
        return False, conflict
```

- This code shows the difference between the current behavior and the expected behavior.

```
>>> from sympy.abc import x
>>> from sympy import ask, Q

>>> ask(Q.gt(x, 0), Q.gt(x, 2))
True # Expected Behaviour, Currently Outputs None

>>> ask(Q.imaginary(x), Q.gt(x, 2))
False # Expected Behaviour, Currently Outputs None
```

- The next thing to be handled is ∞ . Infinity exclusively appears as a boundary limit (e.g., $x < \infty$), not as an algebraic coefficient or variable.
- What we can do is, while preprocessing for `lra_theory`, we can just skip the trivial propositions involving ∞ , something like $x < \infty$ is always True.
- This should lead to this intended behaviour.

```
>>> from sympy import ask, Q, oo
>>> from sympy.abc import x

>>> ask(Q.lt(x, oo), Q.real(x))
True # Expected Behaviour
```

- The final thing to be handled is Irrational numbers. I am not very sure how to fix the issue, but this is something that should be discussed in the Community Bonding Period and addressed properly. I will make sure the mentors know about this. An example of this behaviour is

```
>>> from sympy import ask, Q, pi
>>> from sympy.abc import x

>>> ask(Q.gt(x, pi), Q.lt(x, 3))
False # Expected Behaviour, currently returns None
      # because of UnhandledInput
```

Phase 3: Modernizing the Testing Infrastructure

The SymPy GSoC ideas page explicitly highlights the need to expand Hypothesis (Property-Based Testing) across the codebase, for assumptions, with a similar concern being raised here [Improve testing for assumptions subsystems](#). Currently, the `sympy/assumption-s/tests/` relies almost entirely on static unit tests that might miss edge cases sometimes. This leads regressions later on pointed out by someone or the other. So a way around this is using property based testing with the `Hypothesis` module in Python.

- An example of showing what I intend to do and how it is useful. Take this test as an example in `test_query.py`

```
# From sympy/assumptions/tests/test_query.py
def test_even_query():
    # ... many lines before ...
    assert ask(Q.even(x + y), Q.odd(x) & Q.odd(y)) is True
    assert ask(Q.even(x + y), Q.odd(x) & Q.even(y)) is False
    assert ask(Q.even(x + y), Q.even(x) & Q.even(y)) is True
```

This doesn't cover all the possible options that a user can input, of course.

- This can be tested using Hypothesis testing in such a way (There are few instances where hypothesis testing has been implemented I am following `sympy/polys/tests/test_hypothesis.py` as a reference.

```
from hypothesis import given
import hypothesis.strategies as st
from sympy import ask, Q
from sympy.abc import x, y, z

@st.composite
def temp_strat(draw):
    expr = draw(st.sampled_from([x, y, z,
                                draw(st.integers(min_value=-100, max_value=100))]))

    if draw(st.booleans()):
        expr = expr + draw(st.sampled_from([x, y, z]))
    if draw(st.booleans()):
        expr = expr * draw(st.integers(min_value=1, max_value=5))

    return expr

@given(expr1=temp_strat(), expr2=temp_strat())
def test_even(expr1, expr2):
    assert ask(Q.even(expr1 + expr2), Q.odd(expr1) & Q.odd(expr2)) is True
    assert ask(Q.even(expr1 + expr2), Q.odd(expr1) & Q.even(expr2)) is False
```

- We will try to understand what `temp_strat` does; Basically `draw`, it helps you draw anything randomly from the given range limit first then any random thing from variable or the chosen number.
- Then we again randomly choose if we want to add something more to the `expr` or multiply something to the `expr`.
- Finally we run our test, which runs multiple times, because of the `@given` decorator, every run a new `expr1` and `expr2` is formed and run with the universal truth.

$$ODD + ODD \implies EVEN$$

$$EVEN + ODD \implies ODD$$

As mentioned earlier, both Phase 1 and Phase 2 has a bunch of property based driven test to be written, apart from that I want to integrate it with the existing tests in assumptions module also. The test files which I think to primarily change are:

- `test_query.py`: Multiple ask queries that need to be tested properly. This test is filled with what a user can use ask for.
- `test_rel_queries.py`: Because of the heavy changes in LRA, its better if we have a proper testing of relational queries, so no existing bugs pass.
- `test_satask.py`: The new EUF Logic should not get stuck in some infinite recursion or something of that sort. Newer logic should also be backward compatible with the previous tests and edge-cases.

This is my initial plan for converting the tests using `Hypothesis` Module, during the project period, I will see if more test files are required to be converted or not.

5 Timeline and Milestones

Pre-GSoC Period

Before the contributor results are announced, I would like to work on my 2 open PRs and get them merged. I will work on Tilo's suggestion [lra_theory: Implement more preprocessing steps](#); this will be an initial stepping stone for Phase 2. If we implement this, it will be beneficial for us, as Phase 2 can phase in smoothly from the start. I want to read more on SymPy's existing hypothesis test file and use it in the pre processing PR, this will help me in having a smoother ride in Phase 1's testing the algorithms.

Timeline

- **Community Bonding Period (May 1 - May 24):** As most of my project is highly collaborative, I need to talk properly with Tilo and my fellow mentors with what changes in the project timeline should be essentially done, so that it suits the best. As I am already an avid contributor to SymPy, I mostly don't need this period for setting up my dev environment or learning about other things.
A point to be noted here is though, from May 3 to May 16, I would be having my semester end exams. For this period of time only I would be unavailable as my focus would be there.

The further timeline is assuming the things mentioned above are followed as it is, though I believe a lot of timings will change with different priorities.

- **Week 1-2 (Phase 1)** Check on Krishnav's PRs properly and have discussions with Tilo and successfully implement PR1 (Mentioned in Phase 1).

- **Week 3-4 (Phase 1)** Wire the things to SAT solvers, essentially implement and complete PR2 (Again mentioned in Phase 1).
- **Week 5-6 (Phase 2)** Decouple the legacy `ask` handlers by finalizing the `recursive_ask` helper function thus bypassing the intermediate SAT initializations (PR #29507). Integrate the stalled linear inequality logic (PR #29506).
- **Week 7-8 (Phase 2)** Upgrade the `LRAVariable` class to track complex/real domain states. Have a proper discussion with Tilo about handling ∞ in preprocessing and how to tackle the issue of irrational numbers and then implementing the same and getting it in, making LRA more robust.
- **Week 9-10 (Phase 3)** Build custom `@composite` data generation strategies for SymPy ASTs. Change brittle legacy test suites (specifically `test_query.py` and `test_rel_queries.py`) and convert static assertions into dynamic `Hypothesis` property tests, vastly increasing edge-case coverage.
- **Week 11-12 (Buffer and Documentation):** I really want to keep these two weeks as buffer because the initial phases might end up taking more time, so to properly cover all of the things that I have written. There is also a high chance in Phase 3, a lot of new regressions are discovered, ample amount of time will go in fixing that, thus requiring the buffer period.
A small amount of time to document everything and write it in my final report should also be given in these weeks.

A point was raised by Tilo in [comment](#) about how will I avoid falling into the trap of massive PRs. I have tried to divide my proposal into phases and subphases so that such massive PRs are not created in a single go. Vertically slicing things is beneficial into steps. But if I still end up falling into 1 or 2 big PRs, my first goal should not be to trouble mentors into reviewing things and first judging myself, if this big PR is actually needed or can we cut it down. In the next meet, I can have a discussion with my mentor thus properly agreeing on the structure first and then dividing the existing PR.

Post-GSoC Period

I plan to remain an active contributor to SymPy by smoothing more rough edges in the assumptions module. I assume my project could be the stepping stone for smoothing the assumptions module with these strong mathematical models. I also want to explore the opportunity to add property based testing in other parts of SymPy with help of other maintainers/contributors. I also look forward to mentoring new contributors, as well as reviewing their code in the assumptions module.

6 Commitment

I am willing to commit 30 to 35 hours a week to my project over the 12 week period of GSoC. I don't have any commitments for this summer, so I can work effectively, even putting in more than 30 hours if needed.

If by any chance I get involved somewhere and am not able to achieve a deadline, I will inform my mentors well beforehand and try to compensate as much as I can over the next few weeks.

I plan to utilize the 12 weeks to the utmost, as my project seems ambitious and requires time that I am willing to give.

7 Acknowledgments and References

Acknowledgments

As I am ending my proposal, I want to express my appreciation and acknowledge the immense support from SymPy's community for helping me contribute. The patience and thorough reviews on my initial pull requests have been invaluable for me to grow as a contributor.

I want to extend my special thanks to Tilo RC ([TiloRC](#)) for guiding me through the assumptions module and helping me shape this proposal. I would also like to thank Anutosh Bhat ([@anutosh491](#)), a senior at my college, for initially introducing me to SymPy and encouraging my early steps into open-source. My appreciation to all the collaborators/maintainers that helped me contribute to SymPy is immense.

References

1. [GSoC 2025 Final Report – Krishnav Bajoria : Extending the Assumptions Module and Integrating an EUF Theory Solver](#)
2. [GSoC 2023 Report Tilo Reneau-Cardoso: Improving Relational Assumptions in SymPy's New Assumptions](#)
3. [GSOC 2025 Proposal - Improving Relational Reasoning and Introducing Quantifier Support in SymPy's Assumptions Framework](#)
4. [Tilo RC's Proposal](#)
5. [Anutosh Bhat's Proposal](#)
6. [My discussion with Tilo on Google Group.](#)
7. All other linked PRs and Issues mention in this Proposal.